

Experiment Report: Sentiment Analysis Using Pretrained BERT Model

Name: zain pawle

Roll No: 25dco08

Batch: 3

Class: TECO1

September 9, 2026

1 Aim

To implement a Sentiment Analysis application using a pretrained BERT model (DistilBERT fine-tuned on SST-2) with a FastAPI web backend and HTML/CSS/JS frontend interface.

2 Theory

Bidirectional Encoder Representations from Transformers (BERT) is a transformer-based machine learning framework developed by Google for Natural Language Processing (NLP). Standard sequential models (like LSTMs) read text left-to-right or right-to-left sequentially. In contrast, BERT reads the entire sequence of words at once using bidirectional attention mechanisms.

In this project, DistilBERT—a smaller, faster, and lighter transformer model distilled from BERT—is used. It retains 97% of BERT’s language understanding capabilities while being 60% faster. For sentiment classification, input text is tokenized with Special Tokens ([CLS], [SEP]) and passed through self-attention layers to compute a dense feature representation. A standard linear classification head with a Softmax output layer predicts the sentiment label (POSITIVE or NEGATIVE).

3 Software and Hardware Requirements

3.1 Software Requirements

- **Operating System:** Linux / macOS / Windows
- **Language/Runtime:** Python 3.10+
- **Core Libraries:** fastapi, uvicorn, transformers, torch, pydantic
- **Frontend Stack:** HTML5, CSS3, JavaScript (Fetch API)

3.2 Hardware Requirements

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher
- **RAM:** Minimum 8 GB (16 GB recommended)
- **Storage:** ~ 2 GB free space for deep learning dependencies and model weights
- **GPU:** Optional (NVIDIA CUDA support enabled; CPU execution fully supported)

4 Mathematical Formulae

4.1 Self-Attention Mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Where Q , K , and V are Query, Key, and Value matrices derived from token embeddings, and d_k is the dimensionality of key vectors.

4.2 Softmax Classification Output

$$P(y = c \mid x) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$$

Where z_c is the logit score corresponding to sentiment class $c \in \{\text{POSITIVE}, \text{NEGATIVE}\}$.

5 Libraries Used

- **transformers:** Provides pretrained transformer pipelines.
- **torch:** Deep learning computation framework underlying HuggingFace models.
- **fastapi:** High-performance web framework for constructing APIs.
- **uvicorn:** ASGI web server to execute FastAPI applications.

6 Source Code

6.1 Backend Code (app.py)

```
1 from fastapi import FastAPI, HTTPException
2 from fastapi.staticfiles import StaticFiles
3 from fastapi.responses import FileResponse
4 from pydantic import BaseModel
5 from transformers import pipeline
6
7 app = FastAPI(title="Sentiment Analysis API")
8 classifier = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")
9
10 class TextRequest(BaseModel):
```

```

11     text: str
12
13 app.mount("/static", StaticFiles(directory="static"), name="static")
14
15 @app.get("/")
16 def read_root():
17     return FileResponse("static/index.html")
18
19 @app.post("/api/analyze")
20 def analyze_sentiment(request: TextRequest):
21     if not request.text.strip():
22         raise HTTPException(status_code=400, detail="Text cannot be
empty.")
23     result = classifier(request.text)[0]
24     return {"label": result["label"], "score": round(float(result["
score"]), 4)}

```

7 Output

- **Input 1:** "Implementing sentiment analysis with pretrained BERT model is super easy and efficient!"
- **Result 1:** POSITIVE (Confidence Score: 99.96%)
- **Input 2:** "This service is terrible and disappointing."
- **Result 2:** NEGATIVE (Confidence Score: 99.98%)

8 Conclusion

The Sentiment Analysis web system was successfully implemented using a pretrained DistilBERT transformer model paired with FastAPI and a modern HTML/CSS interface.

Student Information:

- **Name:** zain pawle
- **Roll No:** 25dco08
- **Batch:** 3
- **Class:** TECO1